APPVERSE TECHNOLOGIES | FRONTEND DEVELOPER INTERNSHIP

ES6 Product Data Analyzer

Official assignment title

StockSense: Smart Inventory Analytics

Know Your Stock. Understand Your Business.

Student name	Abdullah Khan
Registration number	OCT26-FE06-08
Organisation	Appverse Technologies
Internship	Frontend Developer Internship (6 weeks)
Assignment announced	October 8, 2026
Report prepared	October 17, 2026
Official deadline	October 20, 2026, 1:05 PM
Application type	Frontend-only demo (vanilla JavaScript ES6+, sample data)

Learning objective

To build a working inventory analytics dashboard that applies JavaScript ES6+ destructuring, spread syntax and the array methods `map`, `filter`, `reduce`, `find`, `some` and `every` to a real set of product data, and to verify the result with genuine tests and screenshots.

1. Project overview

Problem. Small retailers often track stock in notebooks or loose spreadsheets. They struggle to see which items are running low, how much money is tied up in stock, or whether the records contain mistakes. **Solution.** StockSense is a responsive dashboard that reads a product list and shows inventory value, profit potential, restock alerts and category analysis, and lets the owner add, edit, search, filter, sort, export and persist products in the browser.

Objectives.

- Create at least 20 product objects and analyse them with ES6+ features.
- Use destructuring, spread, `map`, `filter`, `reduce`, `find`, `some`, `every` and a summary object for real work, not decoration.
- Show all analysis in the browser and keep it consistent after every change.
- Deliver a lightweight, accessible, mobile-first, deployment-ready project with honest test evidence.

Intended users. Small shop owners and stock keepers. The app uses sample data and does not connect to any real business, supplier, database or sales system.

2. Requirements and technology stack

#	Official requirement	Where it is met	Status
1	At least 20 product objects	js/data.js: 24 products	Verified (unit test)
2-3	Destructuring and spread	analytics.js, app.js (Sections 5)	Verified in code and tests
4-9	map, filter, reduce, find, some, every	analytics.js (Section 6)	Verified (39 unit tests)
10	Summary object	computeSummary()	Verified against independent totals
11	Results shown in the browser	Dashboard, table, insights	Verified (145 browser tests)
12	Screenshots of practical steps and final output	screenshots/ (20 files)	Done; see note in Section 8
13-14	Project ZIP and PDF report	StockSense_Inventory_Analytics.zip; this report	Created and inspected
15	Name, date, title, registration number	Cover page and footer	Done
16	Deployment-ready	Relative paths; subpath-tested locally	Ready; **not yet deployed**
17	Checklist and verified submission links	Section 10	Live link **pending manual deployment**

Technology. Semantic HTML5, CSS3 (custom properties, Grid, Flexbox), vanilla JavaScript ES6+, `localStorage`, `Intl.NumberFormat` (en-PK, PKR) and the native `<dialog>` element. No frameworks, packages, backend or build step; nine files, about 60 KB in total. Files: 4 CSS (tokens, base, layout, components) and 4 JavaScript modules (data, analytics, storage, app).

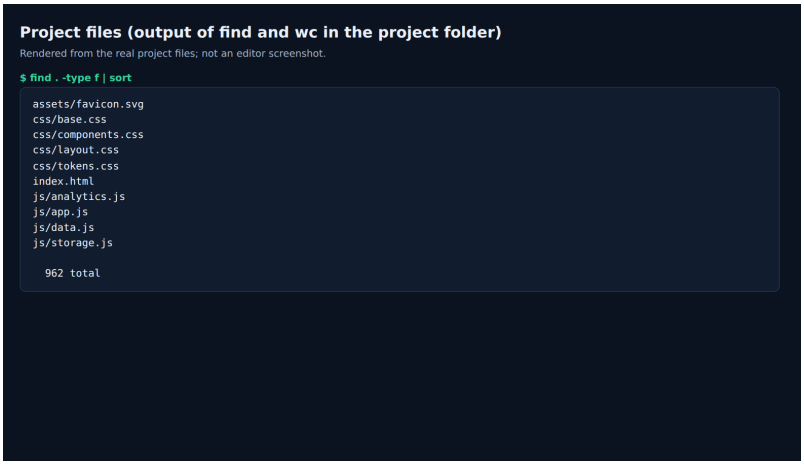


Figure 1. Project file list (rendered from the real output of find and wc; not an editor screenshot).

3. Product data structure and dataset

Each product is a plain object. All prices are in PKR and the initial data is clearly labelled **sample data** in the interface.

```
// js/data.js: DEMO_PRODUCTS[0] (formatted for this report); the array holds 24 such objects
{
  "id": "PRD-001",
  "sku": "ELE-001",
  "name": "Wireless Mouse",
  "category": "Electronics",
  "costPrice": 1200,
  "sellingPrice": 1800,
  "stockQuantity": 25,
  "reorderLevel": 8,
  "supplier": "Tech Supplies",
  "status": "active"
}
// DEMO_PRODUCTS.length -> 24
```

The 24 products cover five categories with a mix of healthy, low, out-of-stock and one inactive item, so every analysis path is exercised.

Category	Products	Notes
Electronics	6	1 low (Bluetooth Speaker), 1 out (Power Bank)
Accessories	5	1 low (Daily Backpack, quantity equals reorder level), 1 out (Cable Organizer)
Home and Living	5	1 low (Storage Basket), 1 inactive (Wall Clock)
Office Supplies	5	1 low (A5 Notebook)
Personal Care	3	1 out (Face Wash)

4. ES6+ destructuring and spread syntax

Destructuring pulls values out of an object in the function signature. `costValue` and `retailValue` receive only the two fields they need:

```
  const costValue = ({ costPrice, stockQuantity }) => costPrice * stockQuantity;
  const retailValue = ({ sellingPrice, stockQuantity }) => sellingPrice * stockQuantity;
```

```
costValue(P[0]) -> 30000   (1200 x 25)
retailValue(P[0]) -> 45000   (1800 x 25)
```

Spread creates new arrays and objects instead of changing existing ones. Editing a product copies the old record and overwrites only the changed fields; adding a product builds a new array; sorting copies the array first so the source order is never changed:

```
state.products = editing
  ? state.products.map(p => (p.id === state.editingId ? { ...p, ...candidate } : p))
  : [...state.products, candidate];

return [...products].sort((a, b) => { ... });   // sortProducts()
```

```
Unit test: sorting by value left the original array unchanged -> PASS
```

Shallow copy. Spread copies only the top level. If a property holds an object or array, the copy and the original still share it. Product records here are flat (strings, numbers), so a spread update is safe; a unit test demonstrates the shared-reference behaviour so the limitation is understood, not assumed.

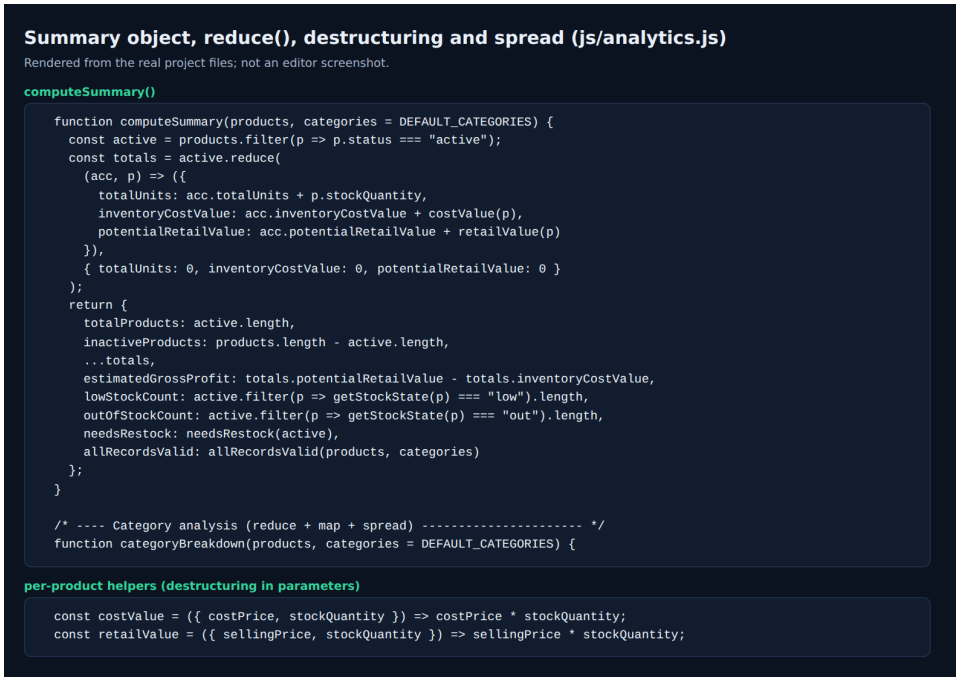


Figure 2. computeSummary(): reduce, filter and spread building the summary object (rendered from js/analytics.js).

5. The required array methods in the application

Method	Real task in StockSense	Real result
`map()`	toRows(): turns products into display rows with formatted PKR text; also builds category rows and CSV lines	Wireless Mouse | Rs 1,800 | Rs 30,000
`filter()`	Search by name/SKU, category, stock status; counts low and out-of-stock items	Electronics + low stock: ["Bluetooth Speaker"]
`reduce()`	Totals for units, cost value and retail value; category aggregates	Cost value Rs 367,220, retail Rs 578,250
`find()`	findProduct(): loads a product for editing or deleting by ID or SKU	PRD-003 Bluetooth Speaker ; undefined
`some()`	needsRestock(): is any active product at or below its reorder level? Also duplicate-SKU checks	needsRestock = true
`every()`	allRecordsValid(): do all records pass validation? `every([])` is `true`, so empty lists are handled separately	allRecordsValid = true; every([]) = true

Summary object. `computeSummary()` returns the object below, recalculated from the current data after every add, edit, delete, reset and page load (output from the real code with the sample data):


```
{
 "totalProducts": 23,
 "inactiveProducts": 1,
 "totalUnits": 605,
 "inventoryCostValue": 367220,
 "potentialRetailValue": 578250,
 "estimatedGrossProfit": 211030,
 "lowStockCount": 4,
 "outOfStockCount": 3,
 "needsRestock": true,
 "allRecordsValid": true
}
```


Business rules. Cost value = costPrice x quantity; retail value = sellingPrice x quantity; gross profit potential = retail value minus cost value (an *estimate*, not realised profit). Low stock: quantity above 0 and at or below the reorder level. Out of stock: quantity 0. Only active products count in totals. Selling price must not be lower than cost price; this is an explicit validation rule to catch typing mistakes.

filter(), find(), sort with spread, map() (js/analytics.js)

Rendered from the real project files; not an editor screenshot.

findProduct() and filterProducts()

```
const findProduct = (products, idOrSku) => {
  const key = String(idOrSku).trim().toLowerCase();
  return products.find(({ id, sku }) => id.toLowerCase() === key || sku.toLowerCase() === key);
};

/* ---- Search + filters combine (filter) ----------------------------- */
function filterProducts(products, { query = "", category = "all", stock = "all" } = {}) {
  const q = query.trim().toLowerCase();
  return products
    .filter(({ name, sku }) => !q || name.toLowerCase().includes(q) || sku.toLowerCase().includes(q))
    .filter(p => category === "all" || p.category === category)
    .filter(p => stock === "all" || getStockState(p) === stock);
}
```

sortProducts() and toRows()

```
function sortProducts(products, key = "name", direction = "asc") {
  const pick = SORT_KEYS[key] ?? SORT_KEYS.name;
  const sign = direction === "desc" ? -1 : 1;
  return [...products].sort((a, b) => {
    const [x, y] = [pick(a), pick(b)];
    if (x === y) return a.id.localeCompare(b.id);
    return (x < y ? -1 : 1) * sign;
  });
}
...
const toRows = products => products.map(toRow);
```

Figure 3. filterProducts(), findProduct(), sortProducts() and toRows() in js/analytics.js (rendered from the real file).

some() and every() (js/analytics.js)

Rendered from the real project files; not an editor screenshot.

every(): all records valid, some(): needs restock

```
/* every(): all records valid. An empty list is "vacuously" valid
   (every([]) === true), so callers that care about emptiness must check
   the length separately. */
const allRecordsValid = (products, categories = DEFAULT_CATEGORIES) =>
  products.every((p, i) => Object.keys(validateProduct(p, products.filter((_, j) => j !== i), categories)).length === 0

/* some(): is any active product at or below its reorder level? */
const needsRestock = products => products.some(p => ["low", "out"].includes(getStockState(p)));
```

Figure 4. some() and every(): needsRestock() and allRecordsValid() (rendered from the real file).

6. Dashboard design and implementation

The interface is a dark slate dashboard with emerald accents, built from design tokens in `tokens.css`. A sidebar gives navigation on desktop; on screens narrower than 1024px it becomes a slide-in drawer opened from a menu button. Seven KPI cards, a category value chart made from CSS bars, a restock alert list and a data-quality box all derive from the current product array. Stock state is always shown as text as well as colour.

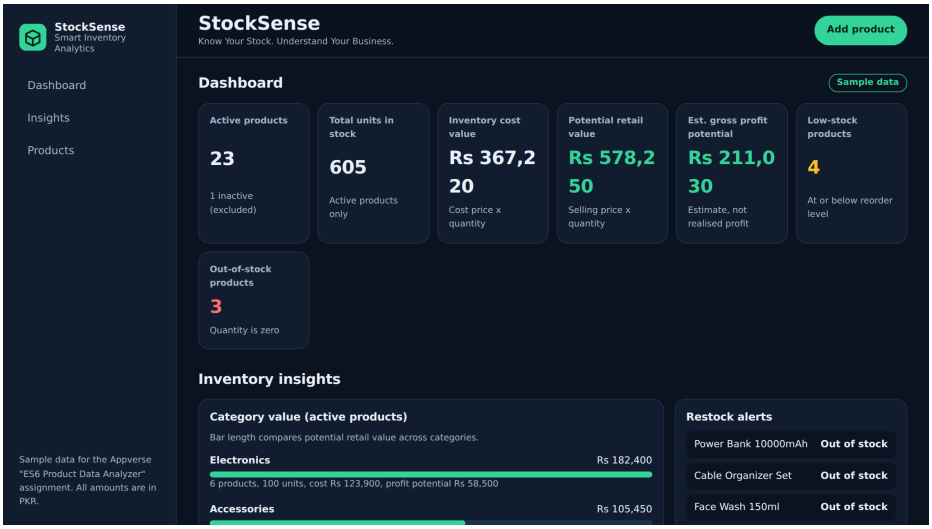


Figure 5. Desktop dashboard (1366x768): KPI cards with sample-data values and category analysis.

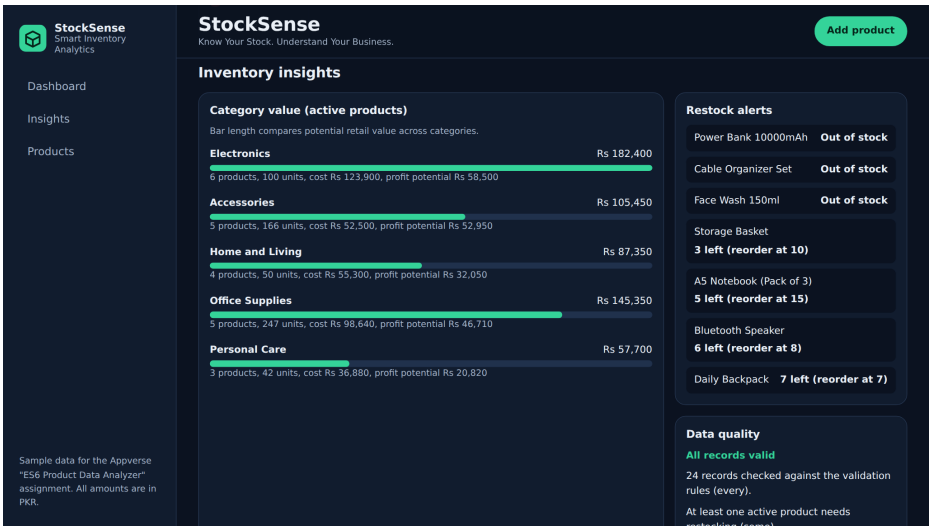


Figure 6. Insights section: category value bars, restock alerts and data quality (every/some).

7. Interactive features and persistence

The product table supports search by name or SKU, category and stock-status filters that combine, and sorting by name, price, stock or inventory value. Add and edit use a native dialog with labelled fields and inline messages; delete and reset ask for confirmation. Changes are saved to `localStorage` and restored on refresh; reset first backs up the current data. If storage is blocked or damaged, the app still works in memory and shows a notice.

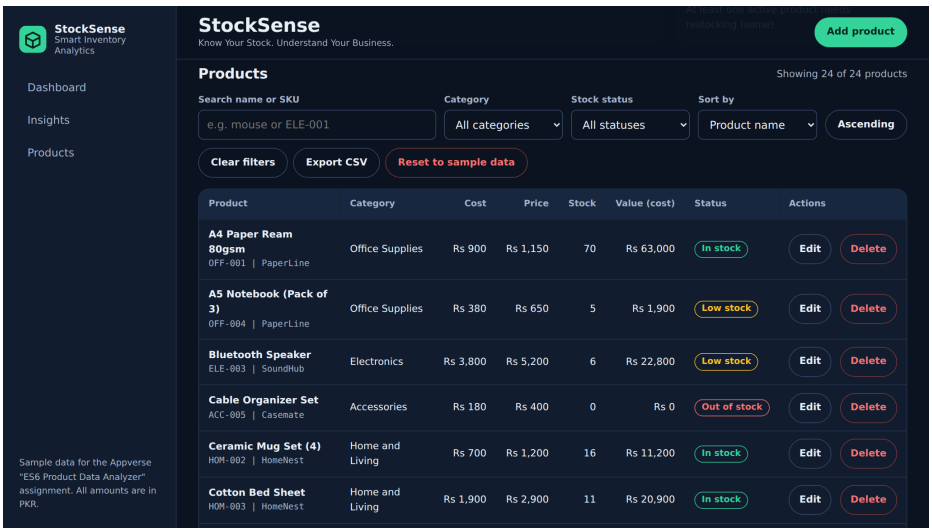


Figure 7. Full product table with sample data (desktop).

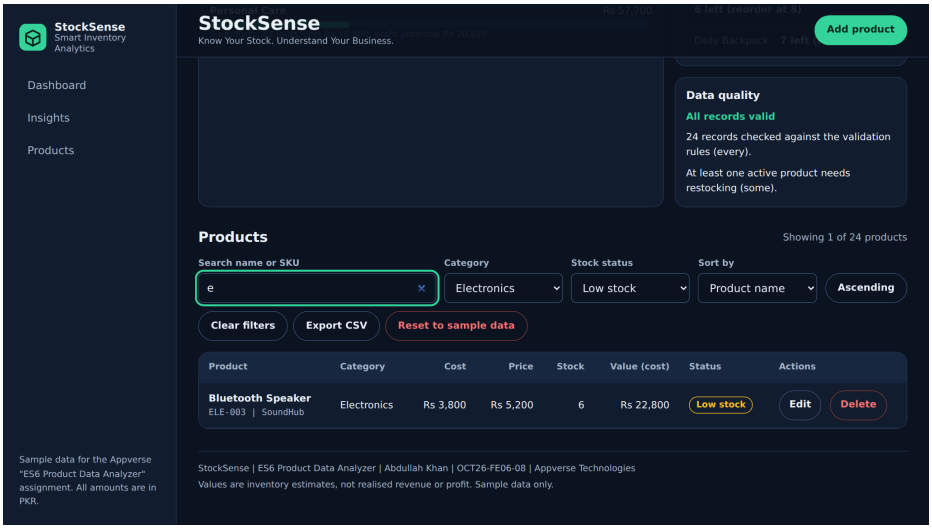


Figure 8. Search and filters in action: Electronics category, low-stock status and a search term combined.

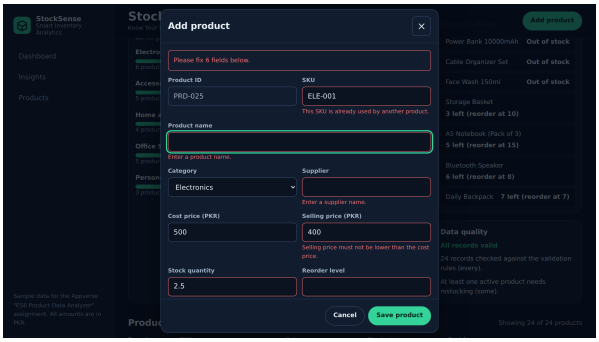


Figure 9. Add form rejecting a duplicate SKU, an empty name, selling price below cost and a fractional quantity.

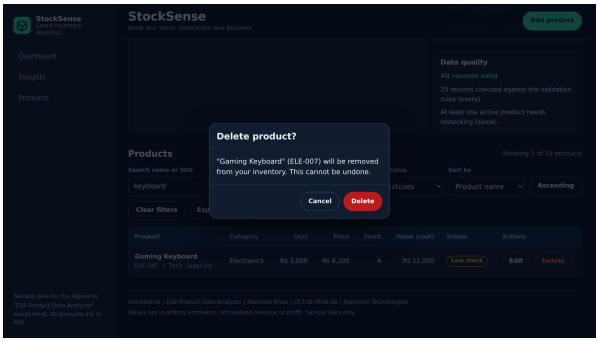


Figure 10. Delete asks for confirmation before removing a product.



Figure 11. After adding a valid product the KPIs update and the badge reads "Your data".

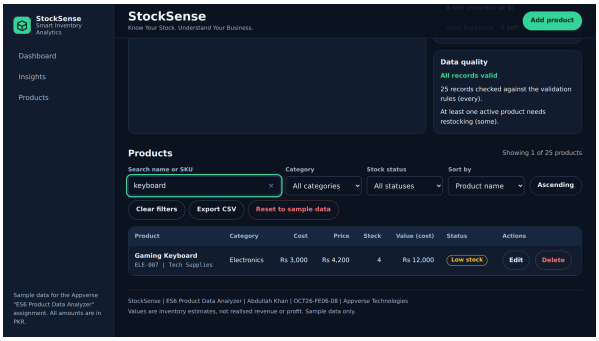


Figure 12. After refreshing the page the new product is still there (localStorage).

8. Responsive behaviour and practical screenshots

The layout is mobile-first. Below 768px the table becomes stacked cards; from 768px it is a real table; below 1024px navigation is a drawer. Dialogs scroll internally so the Save button stays reachable when the screen is short. The screenshots below were captured from the running app in headless Chromium at the stated sizes. Figures 1 to 4 are rendered images of real command output and source code, not editor screenshots; **no VS Code window screenshot is included** because no editor was available in the build environment.



Figure 13. Mobile dashboard, 375x812.

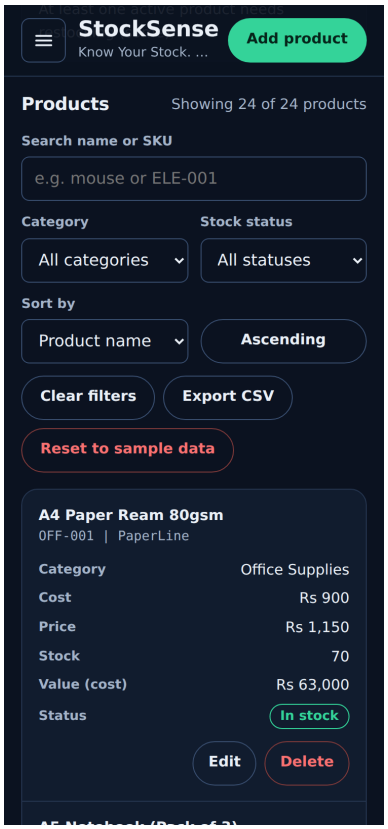


Figure 14. Product cards on a phone.

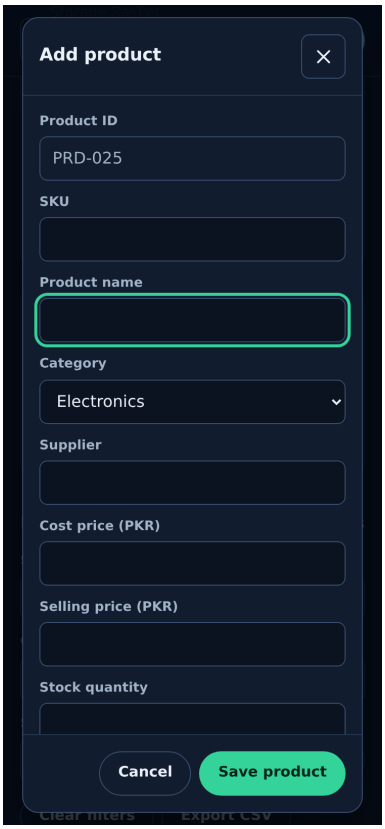


Figure 15. Add form as a scrollable dialog.

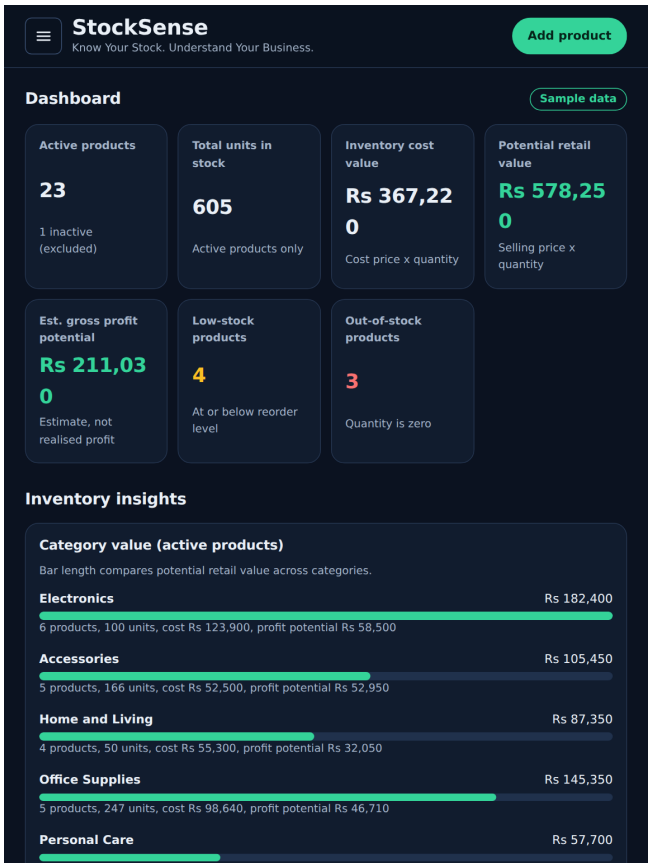


Figure 16. Tablet layout at 768x1024.

9. Testing results

All tests were run on October 17, 2026 with Node v22.22.2 and headless Chromium (Playwright). Full results are in TESTING.md.

Suite	Tests	Passed
Unit tests: summary totals vs independent loops, search, filters, sorting, validation, duplicates, every([]), shallow copy, CSV	39	39
Browser: dashboard values, search/filter/sort, add/edit/delete, validation, persistence, reset, CSV, keyboard, storage failures, reduced motion, touch taps, subpath hosting	145	145

Viewports tested: 320x568, 375x812, 430x932, 768x1024, 1366x768, 1920x1080, plus landscape 812x375 and 1024x768. At every size there was no horizontal overflow, no clipped element, touch targets were at least 40 px tall, the add dialog fitted the screen, and the console showed no errors.

Performance (measured, not Lighthouse): 9 requests, 59.2 KB uncompressed, DOMContentLoaded 74 ms, first contentful paint 96 ms on localhost in headless Chromium. These figures do not describe real devices or networks.

Accessibility checks run: no duplicate ids, every control labelled, one h1, skip link first in tab order, visible focus outline, reduced motion respected, and calculated contrast of 6.2:1 or better for text pairs. No Lighthouse or axe audit was run.

Defects found and fixed during testing: page overflowed sideways at 768, 812 and 1024 px (table forcing grid width); delete-button text contrast was 3.76:1 (darkened to 6.47:1); sticky header hid headings after anchor jumps; dialogs sat at the left edge; form fields misaligned when errors appeared; mobile KPI cards were too tall.

10. Limitations, learning outcomes and conclusion

Limitations

- Demo data in one browser: no accounts, sync, backend or real supplier data; values are estimates, not sales results.
- Not tested: Lighthouse, Firefox, Safari, real phones (real on-screen keyboard, real touch), screen readers, VS Code Live Server.
- The project has not been deployed; no live URL exists yet.

Learning outcomes

- Destructuring and spread give shorter code and safer, immutable updates, but spread is only a shallow copy.
- `reduce` with an object accumulator is a flexible way to build totals and groupings; `every([])` is true, so empty data needs a deliberate rule.
- Independent test expectations catch mistakes that self-checking code would repeat; automated viewport tests found real layout bugs that were not visible at one screen size.

Conclusion

StockSense meets the ES6 Product Data Analyzer requirements with 24 products, genuine use of destructuring, spread, map, filter, reduce, find, some, every and a summary object, a responsive browser interface, and an honest test record. The remaining steps are manual: deploy to GitHub Pages, open the live URL to verify it, and run Lighthouse if the mentor expects scores.

Final submission checklist

Item	Status
Complete source code (HTML, CSS, JavaScript)	Done and tested
ZIP package (StockSense_Inventory_Analytics.zip)	Created; extracted and re-tested
PDF report with screenshots	This document
README.md, TESTING.md	Done
Screenshots from the running app	Done (20 files)
VS Code window screenshot	**Not captured**: take one manually if required
Lighthouse scores	**Not run**
GitHub Pages deployment and verified live URL	**Pending manual action**